CHAPTER 20

Distributed Message Queue

Design a Kafka-like durable, ordered, horizontally scalable log for decoupling services.

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design a distributed message queue (Kafka / RocketMQ / Pulsar style): producers send messages to **topics**, consumers read them. It decouples services, buffers bursts, and supports high throughput with durability. Decide on partitions, replication, consumer groups, delivery semantics and ordering.

Why interviewers ask it: A deep infrastructure question: append-only log, batching, replication (ISR), delivery guarantees, rebalancing.

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
Messages: format and size?	Text, KB-range (up to MB).
Repeated consumption?	Yes, consumers can replay (retain data).
Order of messages?	Preserved within a partition.
Retention?	2 weeks.
Delivery semantics?	At-most-once, at-least-once, exactly-once configurable.
Throughput / latency?	High throughput (log aggregation) and low latency option.

3. Requirements

Functional

- Producers send messages to a topic.
- Consumers subscribe and consume (single or group).
- Messages can be consumed repeatedly (replay).
- Old messages are truncated by retention.
- Delivery semantics configurable.

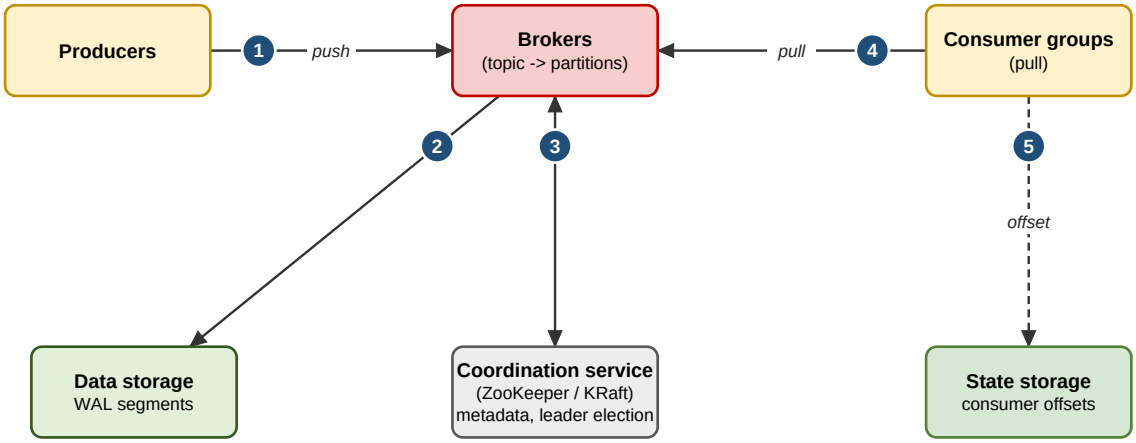
Non-functional

- High throughput (configurable) and low latency.
- Scalable (partitions, brokers).
- Persistent and durable.
- Fault tolerant (replication).

4. Back-of-the-envelope estimation

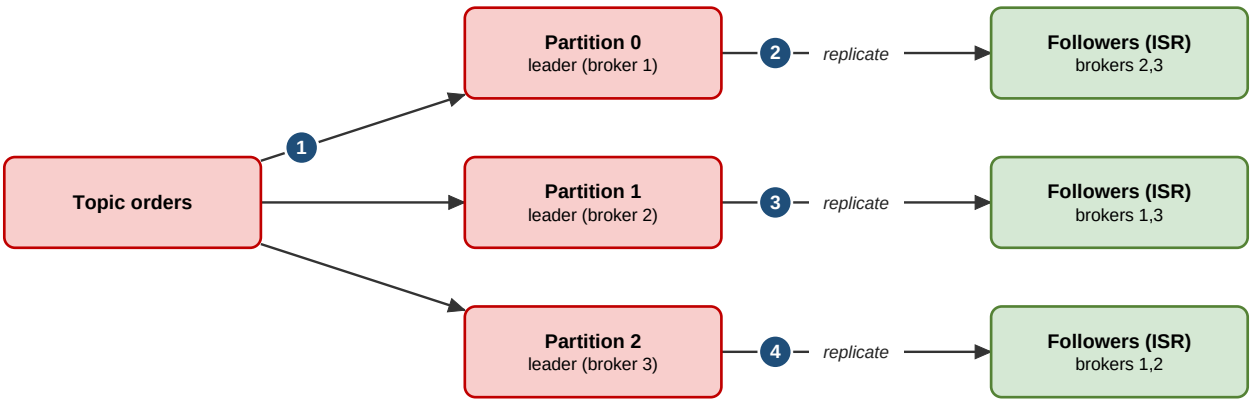
- Assume 1 M msgs/s x 1 KB = 1 GB/s ingest. 2-week retention = 1 GB/s x 1.2 M s = about 1.2 PB; replication x3 = 3.6 PB -> hundreds of brokers with 10-20 TB disks.
- Sequential disk I/O (500 MB/s+ per disk) with OS page cache is why a disk-based design can meet this throughput.

5. High-level design



Step	What happens
1	A producer **pushes** a message (key, value) to a **topic**. The producer batches messages and picks a partition by key hash (so related messages stay ordered).
2	The broker appends the batch to the partition's **write-ahead log** (segment files, sequential I/O) and replicates it to followers; with `acks=all` it is committed once the in-sync replicas have it.
3	Brokers keep metadata (topics, partitions, leaders, membership) in the **coordination service**, which also elects the controller and partition leaders.
4	Consumers in a **consumer group pull** batches from their assigned partitions (one consumer per partition within a group), at their own pace.
5	After processing, consumers **commit offsets** to state storage. On restart or rebalance they resume from the committed offset.

Topic -> partitions -> replicas



Step	What happens
1	A topic is split into **partitions**, spread across brokers for parallelism. Ordering is guaranteed only within one partition.
2	Each partition has one **leader** and several **followers** on other brokers. Followers that are caught up form the **ISR** (in-sync replicas).
3	Leaders are balanced across brokers (broker 1, 2, 3 each lead one partition) so load is even.

Step	What happens
4	If a leader dies, the controller elects a new leader from the ISR, so committed messages are not lost.

6. Deep dive

Data storage: append-only write-ahead log

- Messages written sequentially to **segment files** (e.g. 1 GB each) -> O(1) disk appends are fast; old segments deleted/compacted by retention. Zero-copy transfer (sendfile) and OS page cache for reads.
- Messages: `key, value, topic, partition, offset, timestamp, size, CRC`. Offset = position in the partition.
- Batching everywhere (producer batches, broker disk writes, consumer fetches) is the key throughput trick - trade slight latency for big gains.

Topics, partitions, brokers

- A **topic** is split into **partitions** (shards) spread across brokers. Ordering is guaranteed **only within a partition**; choose partition by key hash (e.g. `user_id`) to keep related messages ordered.
- More partitions = more parallelism but more files/metadata overhead and slower recovery.

Consumer groups

- Each partition is consumed by **at most one consumer in a group**; multiple groups each get all messages. Consumers **pull** (not push): consumer controls rate, batches, no overload; long-poll to avoid empty polls.
- **Rebalancing** on join/leave/crash: a group coordinator (using heartbeats) reassigns partitions; consumers resume from their committed offset. Stored in state storage (offsets).

Replication and ISR

- Each partition has a **leader** and **followers**; producers write to leader; followers replicate. **In-sync replicas (ISR)** = replicas close to the leader. A message is **committed** when all ISR have it (ack = all).
- `acks=0` (fire and forget; lowest latency, may lose), `acks=1` (leader only), `acks=all` (strongest; slowest). Leader failure -> controller elects a new leader from the ISR.

Delivery semantics

- **At-most-once:** producer no retries (acks=0); consumer commits offset before processing. Fits metrics where loss is ok.
- **At-least-once:** acks=all/1 with retries; consumer commits offset after processing -> duplicates possible -> make consumers idempotent. Most common.
- **Exactly-once:** idempotent producer (sequence numbers) + transactions + dedupe at consumer; costly and complex; needed for finance.

Coordination and metadata

- A cluster **controller** (via ZooKeeper / Raft) keeps broker membership, topic/partition metadata, leader election, and ACLs.

Scaling and operations

- Add brokers -> reassign partitions gradually; throttle replication traffic. Handle **slow consumer lag** with monitoring (lag per group), alerting, and autoscaling consumers up to the partition count.
- Message filtering via tags/topics; delay queue with time-wheel for delayed messages; dead-letter topics for poison messages; compaction for changelog topics.

7. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Push vs pull consumers	Push: low latency, can overwhelm / Pull: consumer-paced, batching	Pull with long polling (Kafka standard).
Throughput vs latency	Large batches / small immediate sends	Tunable per topic (linger.ms, batch size).
acks	0 / 1 / all	all for durability; 1 for balance; 0 for telemetry.
Partition count	Few / many	Size to target consumer parallelism with headroom; avoid excess.

8. Scaling, reliability, failure modes

- Horizontal: more partitions and brokers; rack-aware replica placement; multi-AZ replication.
- Tiered storage for older segments (cheap object storage).
- Disaster recovery via cross-cluster mirroring (MirrorMaker).

9. Likely follow-up questions

- How do you guarantee exactly-once processing?
- What happens when the leader of a partition dies mid-write?
- How would you implement delayed messages and message priorities?
- How do you choose the number of partitions for a topic?

10. Common mistakes

- Expecting global ordering across partitions.
- Committing offsets before processing and then claiming at-least-once.
- Ignoring consumer rebalancing and idempotency.

11. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	5 min	"I'll treat this as a Kafka-like log: producers write to topics, consumers read, messages are retained for two weeks and can be replayed, ordering is preserved within a partition, and delivery semantics are configurable. Assume 1 M messages per second of 1 KB, that is 1 GB/s ingest and about 3.6 PB with replication for two weeks."
2. High-level design	10 min	"Brokers hold topics split into partitions, each stored as an append-only log. Producers push, consumers in consumer groups pull and commit offsets. A coordination service handles metadata and leader election. Each partition is replicated to followers."
3. Deep dive	25 min	"For throughput I use sequential disk writes, batching and zero-copy. Ordering is per partition, chosen by key hash. Replication uses a leader and an in-sync replica set; acks=all commits only after the ISR has the data. Consumers pull; rebalancing reassigns partitions and they resume from committed offsets. Delivery semantics: at-most-once commits offset before processing, at-least-once after, and exactly-once needs idempotent producers and transactions."
4. Wrap up	4 min	"Operationally I'd watch consumer lag, under-replicated partitions and disk usage; scale by adding brokers and reassigning partitions; use tiered storage and mirroring for DR. Extensions are delayed messages, dead-letter topics and compaction."

12. 60-second recap / practice checklist

- Append-only partitioned log + replication (leader/ISR) + consumer groups with committed offsets.
- Pull model, batching, sequential I/O, zero copy.
- Know acks levels and the three delivery semantics.

13. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• Apache Kafka design documentation
• Kafka: A distributed messaging system for log processing
• GeeksforGeeks: Message queues